

Community Library Website

Developer Guide/Walkthrough

COSC 360 - Team 18

Team Members: Ye Thway Aung - Su Myat Thwe - Kiichiro Suganuma - Samuel Bueno
Github Link - <https://github.com/yta3216/CommunityLibrary.git>

Tech-Stack

Database	MongoDB with Mongoose ODM
Backend	Node.js 18, Express 4
Frontend	React 18, React Router v6
Authentication	JSON Web Tokens (jsonwebtoken), bcryptjs for password hashing
Charts	Recharts
HTTP Client	Fetch API (custom apiRequest wrapper in client.js)
State management	React Context (AuthContext), local useState/useReducer per page
Real-Time Updates	Server-sent Events using Express via a persistent HTTP connection

File Structure: Backend (backend/)

File/ Folder	Purpose
server.js	Entry point. Connects to MongoDB, loads middleware, and mounts all routers.
sseManager.js	Maintains a Map of room containing sets of connected clients. Exposes subscribe, unsubscribe, and emit.
middleware/auth.js	authRequired and requireRole middleware. Verifies JWT and attaches req.user.
models/User.js	Mongoose schema for Users: username, email, password, role, status
models/Book.js	Mongoose schema for books: isbn, title, author, genre, owner, holder, status, ownerLocked
models/Chat.js	Mongoose schema for chat: embedded messages [] subdocument array
models/Review.js	Mongoose schema for review: book, reviewer, rating, comment
routes/authRouter.js	POST /api/auth/register, /login, /me
routes/bookRouter.js	CRUD for /api/books, plus /availability, /toggle-status
routes/chatRouter.js	GET /api/chats, POST messages, PATCH lend/return
routes/reviewRouter.js	POST/GET /api/reviews, plus /mine, /all, /user/:id
routes/userRouter.js	GET/PATCH /api/users, admin toggle-status, cycle-role, delete
routes/sseRouter.js	GET /api/events. Injects token into query.
controllers/	Thin handlers: validate, call service, return JSON
services/	Business logic: auth, book, chat, review, user, sse

File Structure: Frontend (frontend/src/)

File/ Folder	Purpose
api/	client.js (apiRequest, createEventSource, token storage), books.js, chats.js
components/	Reusable frontend parts
components/charts	All chart components
context/	AuthContext.jsx (global auth state, token bootstrap)
hooks/	Reusables react logic
pages/	All pages of the website
utils/	General helper functions

Features Implemented

The following features were built and are function in the submitted version of the site.

Authentication, Authorization & Sessions
• JWT-based stateless authentication with a 7-day token expiry. • Secure password hashing using bcrypt. Passwords are never returned in API responses. • Session persistence via localStorage. Stored token is validated server-side on each page load via GET/api/auth/me. • 3 React route guards: RequireAuth (block unauthenticated access), RequireAdmin (blocks non-admin access) and PublicOnly (redirects logged-in users away from login/register).
Book Management
• Full CRUD for book listings: create, read, update and delete • Owner enforcement: only the user who listed the book can edit or delete it. • Book availability is based on holder field: when holder equals owner the book is available if otherwise its current loaned out. • Owners can manually lock their book via PATCH /api/books/:id/availability. • Admin overrides availability changes outside the normal flow.
Borrow & Return Flow
• Borrow requests are initiated through chat system via POST /api/chats/messages. sendFirstMessage validates: the book must be available, the requester cannot be the owner, and the requester cannot already hold another copy of the same ISBN. • The owner confirms lend via POST/api/chats/:id/lend. This sets book.holder to the requester. • The holder confirms return via POST/api/chats/:id/return. This sets book.holder to the owner. • Admin can override a return without the chat flow.
Chat System
• Persistent chat stored as chat documents in MongoDB, each associated with a specific book and pair of users. • Any user can send messages after initial request; full chat history is retained even after the book is returned.
Reviews
• Users can post a review on a book listing that is not theirs. • User can view their review history via GET /api/reviews/mine which is displayed in “My Library” page.
User Profile/ Dashboard & Admin
• Profile editing includes display name and profile picture. Images are stored as ... • User “My library” using Recharts with average book ratings and available/unavailable books based on user listing. Also includes tables with breakdown on all relevant book and user information. • The RequireAdmin component in App.jsx redirects non-admins away from /admin/* routes • Admin panel with full user management: suspend/activate, role cycling, and cascade delete (delete all books owned by user if user deleted) • Admin reports page using Rechart with average ratings, book status breakdowns, and user increase over selected period intervals.

How the site works at a high level

- When a page is loaded, AuthContext validates the stored JWT with GET /api/auth/me and handles global auth state. Unauthorized users are sent back to the login page/unregistered home, and authenticated users are sent to /home/.

- Book listings are fetched on initial mount and kept live via SSE. Searching uses a query sent to GET /api/books/?q= with a debounce of 300 ms.
- Borrowing a book requires sending a message request (via POST /api/chats/messages) which creates a Chat document that stores the book, owner, and the requestor. If the owner chooses to lend the book, it sends a PATCH request to /api/chats/:id/lend to set the book's holder field to the requestor. The requestor can also return the book via PATCH /api/chats/:id/return.
- All mutations emit SSE events to their respective rooms so that all clients viewing a page (and are connected to the relevant room) update without a page refresh.
- Admins access a separate panel at /admin/home, which also broadcast events to public pages via the same rooms. This is guarded by RequireAdmin.

Testing

- The project currently uses automated tests for both backend API and frontend UI .
- Backend tests focus on API contract, authentication/authorization boundaries, and critical book operations.
- Frontend tests focus on user-visible behavior for authentication forms and core book pages (home and detail), using React Testing Library.

Backend covered areas	Frontend covered areas
- Register success and validation failures - Login success and invalid credential handling - Suspended-user login rejection - Book list retrieval success - Book detail retrieval success (authenticated user) - Book detail not found - Book update forbidden for non-owner - Book creation flow, successful authenticated creation and rejection	- Login/Register form rendering and key interactions - Successful and failed book-list display on logged-in home - Successful and failed book-detail rendering - Book creation form behavior
	How to run tests
	- Backend: cd backend → npm test - Frontend: cd frontend → npm test

Known Limitations

The following represent constraints acknowledge by the group:

- Profile images stored as base64: Profile pictures are encoded as base64 data URLs and stored directly in the User document. This keeps the deployment simple (no separate object-storage service required) but increases document size for users who upload high-resolution images, specially in the free version of MongoDB.
 - Furthermore this was also the reason why we decided to not include book images as we feared that the storage available could be quickly used.
- No in-app password change: Passwords are hashed at registration and the current profile-edit flow does not expose a password-change field. If the website were to actually be deployed we would add a dedicated change-password function.
- Cannot modify reviews: Users can only see their review history but cannot delete or modify their review. Admin can delete reviews.